

Practical Fine-Tuning Assignment

Build a Domain-Specific AI Assistant using Unsloth

Objective

In this assignment, you need to build a practical domain-specific AI assistant by fine-tuning an open-source Large Language Model using **Unsloth**.

The goal is to understand a real industry-style fine-tuning workflow:

- Raw domain data preparation
 - Non-instruction fine-tuning
 - Instruction dataset preparation
 - LoRA /DoRA/ QLoRA fine-tuning
 - Preference dataset creation
 - DPO/ORPO alignment
 - Model evaluation
 - Before-vs-after comparison
 - Saving and using the fine-tuned model
-

Business Problem

You are working as a **GenAI Engineer**.

Your company wants to build an internal AI assistant that can answer questions for one specific domain.

The assistant should understand domain-specific terminology, answer user questions clearly, and give better responses than the base model.

Choose Any One Domain

Choose one domain:

1. HR Policy Assistant
 2. Customer Support Assistant
 3. Finance FAQ Assistant
 4. Healthcare FAQ Assistant
 5. Legal Document FAQ Assistant
 6. Course Doubt Assistant
 7. E-commerce Product Support Assistant
 8. IT Helpdesk Assistant
-

Main Task

You need to fine-tune an open-source LLM using **Unsloth** in three stages:

Base Model



Stage 1: Non-Instruction Fine-Tuning



Stage 2: Instruction Fine-Tuning



Stage 3: DPO/ORPO Preference Alignment



Final Domain-Specific AI Assistant

Step 1: Select Domain

Choose one domain and define the business problem clearly.

Example:

If you choose **HR Policy Assistant**, the assistant should answer questions related to:

- Leave policy
- Attendance
- Reimbursement
- Work from home
- Onboarding

- Notice period
- Employee benefits

If you choose **Customer Support Assistant**, the assistant should answer questions related to:

- Refund
 - Order tracking
 - Product issues
 - Cancellation
 - Replacement
 - Payment issues
-

Step 2: Create Non-Instruction Dataset

First, prepare raw domain text.

This is used for **non-instruction fine-tuning**, where the model learns domain language, terminology, writing style, and background knowledge.

Minimum Requirement

Create at least:

- 50 paragraphs of raw domain text

The raw text can come from:

- Public policy documents
 - Public PDFs
 - Product manuals
 - FAQ pages
 - Course notes
 - Legal documents
 - Healthcare information pages
 - IT helpdesk documentation
 - AI-generated domain content that you clean and verify
-

Example Raw Text

Employees are eligible for 12 casual leaves and 10 sick leaves per year. Sick leave must be applied through the HR portal. For leave longer than three days, medical documentation may be required.

Deliverable

data/non_instruction_data.txt/.docx/.pdf

Step 3: Perform Non-Instruction Fine-Tuning

Use your raw domain text to perform a small non-instruction fine-tuning experiment.

The goal is to adapt the base model to the domain before instruction tuning.

Your Notebook Should Include

1. Loading raw domain text
 2. Cleaning and chunking text
 3. Loading base model using Unsloth
 4. Applying LoRA / QLoRA
 5. Training on raw text
 6. Saving the adapter / model
 7. Testing the model after non-instruction fine-tuning
-

Deliverable

notebooks/non_instruction_finetuning.ipynb

Step 4: Create Instruction Dataset

Now create an instruction-response dataset for your selected domain.

This dataset teaches the model how to answer user questions.

Minimum Requirement

Create at least:

- 100 instruction-response examples

Each example should contain:

- instruction / question
 - response / answer
-

Example Format

```
{  
  "instruction": "How can I apply for sick leave?",  
  "response": "You can apply for sick leave through the HR portal by selecting the sick leave  
option and submitting the required details."  
}
```

You can create the dataset manually, from public documents, from public datasets, or with the help of AI.

But you must clean and verify the dataset before using it.

Deliverable

data/instruction_dataset.jsonl

Step 5: Test the Base Model

Before instruction fine-tuning, test the original base model on 10 domain-specific questions.

The goal is to understand how the base model performs before training.

Example

Question: How can I apply for sick leave?

Base Model Answer:

You should contact your employer or HR department for sick leave.

Problem:

The answer is generic and not specific to our HR policy.

Create a Table

Question	Base Model Answer	Problem
----------	-------------------	---------

Deliverable

reports/base_model_evaluation.md

Step 6: Perform Instruction Fine-Tuning using Unsloth

Now fine-tune the model using the instruction-response dataset.

You can continue from the non-instruction fine-tuned model or use the saved adapter/model from Stage 1 as the starting point.

Recommended Models

You can use any one of the following models:

- TinyLlama-1.1B
- Qwen2.5-0.5B
- Qwen2.5-1.5B
- Llama-3.2-1B
- Gemma small model, if available

Your Notebook Should Include

1. Installing required libraries
2. Loading tokenizer
3. Loading model
4. Formatting instruction dataset
5. Applying LoRA / QLoRA
6. Training the model
7. Saving adapter / model
8. Running inference after training

Deliverable

notebooks/instruction_finetuning.ipynb

Step 7: Compare Base Model vs Instruction Fine-Tuned Model

After instruction fine-tuning, ask the same 10 questions again.

Create a comparison table.

Comparison Table

Question	Base Model Answer	Fine-Tuned Model Answer	Which is Better?	Reason
----------	-------------------	-------------------------	------------------	--------

Evaluation Criteria

Evaluate based on:

- Correctness
 - Domain accuracy
 - Clarity
 - Safety
 - Helpfulness
 - Less generic response
 - Better domain-specific behavior
-

Deliverable

reports/sft_model_comparison.md

Step 8: Create Preference Dataset for DPO

Now create a preference dataset for alignment.

This dataset teaches the model which answer is better and which answer is weaker.

Minimum Requirement

Create at least:

- 50 preference examples

Each example should contain:

- prompt
 - chosen response
 - rejected response
-

Example Format

```
{  
  "prompt": "How can I cancel my order?",  
  "chosen": "You can cancel your order from the Orders section before it is shipped. If the order  
has already been shipped, you can request a return after delivery.",  
  "rejected": "Just ignore the order."  
}
```

Guidelines

The chosen response should be:

- Correct
- Helpful
- Safe
- Professional
- Domain-specific

The rejected response can be:

- Wrong
 - Incomplete
 - Unsafe
 - Rude
 - Too generic
 - Not domain-specific
-

Deliverable

data/preference_dataset.jsonl

Step 9: Perform DPO/ORPO Preference Alignment

Use your preference dataset and perform DPO alignment.

The goal is to improve the model's response quality after instruction fine-tuning.

Your Notebook Should Include

1. Loading the SFT model
 2. Loading preference dataset
 3. Formatting prompt, chosen, and rejected responses
 4. Configuring DPO training
 5. Running DPO alignment
 6. Saving the DPO-aligned model
 7. Testing the model after DPO
-

Deliverable

notebooks/dpo_alignment.ipynb

Step 10: Final Evaluation

Now compare all three stages:

1. Base model
 2. Instruction fine-tuned model
 3. DPO-aligned model
-

Final Comparison Table

Question	Base Model Answer	SFT Model Answer	DPO Model Answer	Best Answer	Reason
----------	-------------------	------------------	------------------	-------------	--------

Evaluation Criteria

Evaluate based on:

- Correctness
 - Helpfulness
 - Domain accuracy
 - Safety
 - Tone
 - Clarity
 - Hallucination reduction
 - Professional response quality
-

Deliverable

reports/final_evaluation.md

Step 11: Explain LoRA, QLoRA, Non-Instruction FT, SFT, and DPO/OPRO

Write a short explanation in your own words.

Cover These Points

1. Why full fine-tuning is expensive
2. What LoRA does
3. What QLoRA does
4. Why QLoRA is useful on limited GPU
5. What is non-instruction fine-tuning?
6. What is instruction fine-tuning?

7. What is DPO?
 8. Difference between SFT and DPO
 9. What values you used for:
 - rank
 - alpha
 - dropout
 - learning rate
 - batch size
-

Deliverable

reports/fine_tuning_explanation.md

Step 12: Create a Simple Inference Script

Create a simple Python script where a user can ask a question and your final DPO-aligned model gives an answer.

Example

```
question = "How can I apply for reimbursement?"
answer = generate_answer(question)
print(answer)
```

Deliverable

src/inference.py

Final GitHub Structure

Your repository should look like this:

domain-ai-assistant-finetuning/

```
|
|— data/
|   |— non_instruction_data.txt
|   |— instruction_dataset.jsonl
|   |— preference_dataset.jsonl
|
|— notebooks/
|   |— non_instruction_finetuning.ipynb
|   |— instruction_finetuning.ipynb
|   |— dpo_alignment.ipynb
|
|— reports/
|   |— base_model_evaluation.md
|   |— sft_model_comparison.md
|   |— final_evaluation.md
|   |— fine_tuning_explanation.md
|
|— src/
|   |— inference.py
|
|— README.md
|— requirements.txt
```

README Must Include

Your README should clearly explain:

1. Project title
2. Domain selected
3. Business problem
4. Dataset details
5. Base model used
6. Non-instruction fine-tuning approach
7. Instruction fine-tuning approach
8. DPO alignment approach
9. LoRA / QLoRA configuration
10. Training screenshots or logs
11. Before vs after output comparison
12. Final observations
13. Challenges faced

14. Future improvements

Public Dataset References

Students can either create their own dataset or use public datasets from Hugging Face, Kaggle, or public documents.

Before using any dataset, students must:

- Check the dataset license
 - Clean the dataset
 - Verify the quality
 - Convert it into the required format
-

Customer Support Assistant

<https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset>

Finance / Banking FAQ Assistant

<https://huggingface.co/datasets/PolyAI/banking77>

Healthcare FAQ Assistant

https://huggingface.co/datasets/medalpaca/medical_meadow_medical_flashcards

Legal Document FAQ Assistant

<https://www.atticusprojectai.org/cuad>

<https://arxiv.org/abs/2103.06268>

Course Doubt Assistant / General Q&A

<https://huggingface.co/datasets/toughdata/quora-question-answer-dataset>

E-commerce Product Support Assistant

<https://huggingface.co/datasets/bitext/Bitext-customer-support-llm-chatbot-training-dataset>

<https://arxiv.org/abs/1908.04364>

IT Helpdesk Assistant

<https://www.kaggle.com/datasets/suraj520/customer-support-ticket-dataset>

<https://huggingface.co/datasets/Tobi-Bueck/customer-support-tickets>

HR Policy Assistant

Ready-made HR fine-tuning datasets are limited. The best approach is to use a public employee handbook or HR policy PDF and generate Q&A pairs from it.

Final Expected Output

By the end of this assignment, you should have a working domain-specific fine-tuned LLM that has gone through:

1. Non-instruction fine-tuning
2. Instruction fine-tuning
3. DPO preference alignment

This project should be good enough to explain in interviews as:

I created a domain-specific AI assistant using Unsloth. I first performed non-instruction fine-tuning on raw domain text, then instruction fine-tuning on Q&A data, and finally DPO alignment using preference data. I compared the base model, SFT model, and DPO-aligned model and built a simple inference script.

Submission Instructions

Submit your GitHub repository link.

Your repository must contain:

- Dataset files
- Non-instruction fine-tuning notebook
- Instruction fine-tuning notebook
- DPO alignment notebook
- Evaluation reports
- Inference script
- README file

Make sure your README is clean and easy to understand.

Your project should clearly show:

- What problem you solved
- What model you used
- How you prepared the datasets
- How you fine-tuned the model
- How DPO improved the response quality
- How the final model improved compared to the base model

Submission form:

https://docs.google.com/forms/d/e/1FAIpQLSdDnOZPorHk_bHskQbs9h0IZ1JNErdMXNTBPgns6HHBLfLy9w/viewform?usp=sharing&oid=107026694704731352726

Submission date: till 11 july 2026

Demonstration day: 12 july 2026

You will only select top 3 winner and these three winner will get a access of the upcoming course: <https://www.krishnaik.in/liveclass2/Advanded-Route?id=12>

Use any coding assistant: cluade/codex/coploit/cursor/antigravity